

LOW LEVEL DESIGN (LLD)

CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
2	System Architecture and Component Design	9
3	Apache Kafka Integration	16
4	PSP Switch Low Level Design	22
5	NPCI Router Low Level Design	27
6	CBS Service Low Level Design	32
7	PSP Ledger Service Low Level Design	38
8	NPCI Ledger Service Low Level Design	43
9	Bank Ledger Service Low Level Design	48
10	Audit Service Low Level Design	53
11	Redis Integration	59
12	REST API Implementation	61
13	Virtual Machine Deployment	66
14	Configuration Management	68
15	Exception Handling and Logging	70
16	Performance and Scalability	72
17	Security Considerations	74
18	Testing Strategy	75
19	Future Enhancements	77
20	Conclusion	78

CHAPTER 1 - INTRODUCTION

1.1 Document Information

Field	Value
Project Name	APEX UPI – High Concurrency PSP Switch & Ledger Engine
Document	Low Level Design (LLD)
Backend	Spring Boot
Database	PostgreSQL
Messaging	Apache Kafka
Containerization	Docker
Cache	Redis
Load Testing	Apache JMeter

1.2 Purpose

The purpose of this Low Level Design (LLD) document is to describe the internal implementation details of the APEX UPI system. Unlike the High Level Design (HLD), which explains the overall architecture and interactions between major components, the LLD focuses on the internal design of each microservice, including its classes, packages, Kafka interactions, database operations, REST APIs, configuration, and transaction processing logic.

This document serves as a technical reference for developers, testers, and maintainers. It explains how each service is implemented, how data flows through the system, how Kafka events are produced and consumed, and how transaction information is persisted in PostgreSQL.

The document also describes the implementation of the newly introduced Ledger Engine and Audit Service, which extend the original APEX UPI platform by maintaining transaction ledgers and a centralized audit trail.

1.3 Scope

The scope of this document includes the detailed design of all backend services participating in the transaction lifecycle.

The following services are covered:

- PSP Switch
- NPCI Router
- CBS Service
- PSP Ledger Service
- NPCI Ledger Service
- Bank Ledger Service
- Audit Service
- Dashboard Integration (event consumption and visualization)
- Kafka Messaging Infrastructure
- PostgreSQL Database Design
- Docker Deployment
- Redis Cache Usage
- Configuration Management

The following topics are outside the scope of this document:

- Front-end UI implementation details
- Mobile application development
- Production infrastructure provisioning
- Security certificates and deployment secrets

1.4 Objective

The primary objectives of the Low Level Design are:

- Describe the implementation of each microservice.
- Define the responsibilities of every service.
- Explain package organization and project structure.
- Document Kafka producers and consumers.
- Define REST endpoints.
- Describe PostgreSQL schema and persistence.
- Explain transaction lifecycle.
- Describe event-driven communication.
- Document error handling and transaction recovery.
- Provide implementation details for future maintenance and extension.

1.5 Intended Audience

This document is intended for:

- Software Developers
- Backend Engineers
- DevOps Engineers
- Database Administrators
- QA/Test Engineers
- Technical Reviewers
- Project Mentors

1.6 Technology Stack

Component	Technology
Programming Language	Java 21
Backend Framework	Spring Boot
ORM	Spring Data JPA
Database	PostgreSQL
Messaging	Apache Kafka
Serialization	JSON (Jackson ObjectMapper)
Build Tool	Maven
Containerization	Docker
Cache	Redis
Testing	JMeter
Version Control	Git
Dashboard	HTML, CSS, JavaScript, WebSocket

1.7 Design Principles

The implementation follows a microservice-based architecture. Each service is responsible for a single business capability and communicates asynchronously using Kafka.

The design follows the following principles:

Single Responsibility Principle

Each service performs only one primary responsibility.

Examples:

- PSP Switch initiates transactions.
- NPCI Router validates and routes transactions.
- CBS Service performs banking operations.
- Ledger Services persist financial records.
- Audit Service records transaction history.

Event Driven Architecture

Services communicate through Kafka topics rather than direct synchronous calls wherever appropriate.

Advantages:

- Loose coupling
- Independent deployment
- Scalability
- Fault tolerance
- Easier monitoring

Asynchronous Communication

Instead of directly invoking downstream services, events are published to Kafka.

Benefits include:

- Non-blocking processing
- High throughput
- Retry capability
- Event replay
- Independent scaling

1.8 Architectural Style

The system follows Event-Driven Microservices Architecture.

Client



PSP Switch



Kafka



NPCI Router



Kafka



CBS Service



Kafka



Ledger Services



Audit Service



PostgreSQL

- Each service is independently deployable.
- Each service owns its own business logic.
- Communication occurs through Kafka topics.

1.9 Transaction Overview

A UPI payment follows these processing stages.

Stage 1

Customer initiates payment.

Example:

Alice → Bob

₹500

The PSP Switch receives the payment request.

Stage 2

The PSP Switch validates the request.

If valid:

Kafka Topic

upi.transactions.initiated

is published.

Stage 3

NPCI Router consumes the event.

NPCI performs:

- Bank identification
- Routing validation
- Duplicate transaction checks
- Status update

If successful:

upi.npci.verified

is published.

Stage 4

CBS Service consumes the verification event.

Debit operation is executed.

If successful:

upi.cbs.debit.confirm

is published.

Stage 5

CBS credits the beneficiary account.

After success:

upi.cbs.credit.confirm

is published.

Stage 6

Ledger Services consume Kafka events.

Three ledgers are maintained.

PSP Ledger

Stores initiation details.

NPCI Ledger

Stores routing details.

Bank Ledger

Stores accounting entries.

Stage 7

Audit Service listens to every transaction event.

It stores:

- Initiation
- Verification
- Debit
- Credit
- Reversal

into a centralized audit database.

1.10 Project Modules

The project consists of the following logical modules.

Module	Responsibility
PSP Switch	Accepts payment requests and publishes transaction events
NPCI Router	Validates and routes transactions to the appropriate bank
CBS Service	Performs debit, credit, and reversal operations
PSP Ledger Service	Records payment initiation events
NPCI Ledger Service	Records transaction routing and verification events
Bank Ledger Service	Records debit and credit accounting entries
Audit Service	Maintains the complete transaction audit trail
Dashboard	Displays real-time transaction events and system health
Kafka	Asynchronous messaging infrastructure
PostgreSQL	Persistent storage for ledger and audit data
Redis	Temporary cache for frequently accessed or short-lived transaction data (where applicable in the project)

CHAPTER 2 - SYSTEM ARCHITECTURE AND COMPONENTS DESIGN

2.1 Introduction

The APEX UPI system follows a distributed microservices architecture where each business capability is implemented as an independent Spring Boot service. The services communicate asynchronously using Apache Kafka, allowing the system to process large volumes of transactions with minimal coupling between components.

Unlike a monolithic application where all business logic resides within a single process, this architecture divides the payment lifecycle into multiple independent services. Each service performs a specific responsibility and communicates with downstream services by publishing or consuming Kafka events.

The deployment environment consists of multiple Virtual Machines (VMs), where each VM hosts one or more related services. This separation improves scalability, fault isolation, maintainability, and allows independent deployment and monitoring of individual components.

2.2 Overall Architecture

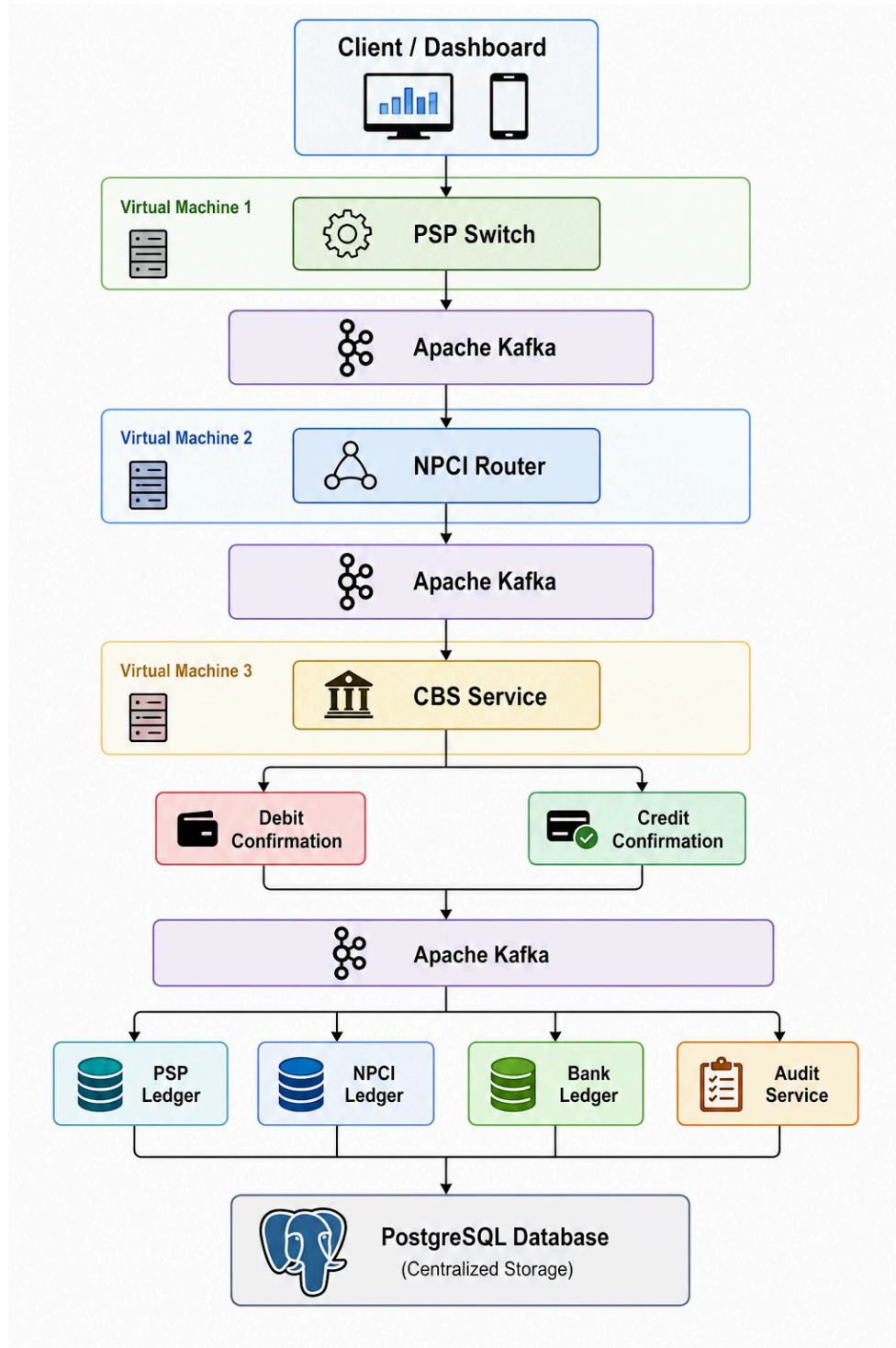
The APEX UPI platform consists of multiple microservices that collectively process a UPI payment from initiation to completion. The system follows an event-driven architecture where Kafka serves as the messaging backbone.

The transaction begins when a user initiates a payment through the PSP Switch. The PSP Switch validates the request and publishes a transaction initiation event to Kafka. The NPCI Router consumes this event, performs routing and verification, and forwards the transaction to the CBS Service. The CBS Service performs debit and credit operations and publishes confirmation events. Ledger Services and the Audit Service consume these events independently and persist transaction information into PostgreSQL.

The architecture ensures that each service operates independently without requiring synchronous communication, thereby reducing service dependencies and improving system resilience.

2.3 Deployment Architecture

The project is deployed using multiple Virtual Machines. Each VM hosts a dedicated business component responsible for a specific stage of transaction processing.



2.4 Microservice Design Philosophy

Each microservice follows the Single Responsibility Principle (SRP). A service is responsible for one business function and owns its corresponding business logic.

The architecture provides the following advantages:

- Independent deployment of services
- Better scalability
- Easier maintenance
- Fault isolation
- Simplified testing
- High throughput through asynchronous messaging
- Loose coupling between services

Every service exposes only the functionality required for its business responsibility while communicating with other services through Kafka topics.

2.5 Service Responsibilities

PSP Switch

The PSP Switch acts as the entry point of the payment system. It receives payment requests from clients through REST APIs. After validating the incoming request, it generates a unique transaction identifier and publishes the payment initiation event to Kafka. It does not directly communicate with the banking system, thereby reducing coupling between services.

Primary responsibilities include:

- Accept payment requests
- Validate request data
- Generate transaction ID
- Publish transaction initiation event
- Return acknowledgement to the client

NPCI Router

The NPCI Router functions as the central routing component. It consumes payment initiation events from Kafka and validates the transaction before routing it to the appropriate banking service.

Its responsibilities include:

- Validate transaction
- Identify payer and beneficiary banks
- Route transaction to CBS Service
- Detect failures
- Trigger automatic reversal when required

CBS Service

The CBS (Core Banking System) Service performs banking operations.

Its responsibilities include:

- Debit payer account
- Credit beneficiary account
- Handle reversal transactions
- Publish debit confirmation
- Publish credit confirmation
- Update banking database

The CBS Service is the only component responsible for modifying customer account balances.

PSP Ledger Service

The PSP Ledger Service stores payment initiation information.

Responsibilities:

- Consume PSP transaction events
- Store initiation details
- Maintain PSP transaction history
- Provide transaction traceability

NPCI Ledger Service

The NPCI Ledger Service records routing and verification information.

Responsibilities:

- Consume NPCI verification events
- Store routing information
- Maintain verification history

Bank Ledger Service

The Bank Ledger Service maintains accounting entries generated by the CBS Service.

Responsibilities:

- Consume debit confirmation events
- Consume credit confirmation events

- Store debit entries
- Store credit entries
- Maintain accounting history

The service follows double-entry accounting principles by recording both debit and credit entries for every successful transaction.

Audit Service

The Audit Service acts as a centralized transaction monitoring system.

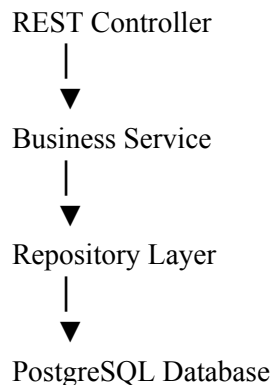
Responsibilities:

- Listen to all transaction events
- Store complete transaction lifecycle
- Maintain audit trail
- Record failures
- Record reversals
- Support compliance and debugging

Unlike Ledger Services, which maintain business records, the Audit Service maintains operational logs.

2.6 Internal Structure of a Microservice

Each Spring Boot microservice follows the same layered architecture.



- The Controller layer handles HTTP requests.
- The Service layer contains business logic.
- The Repository layer interacts with PostgreSQL using Spring Data JPA.
- The Entity layer maps Java objects to relational tables.
- This architecture improves maintainability by separating presentation, business, and persistence logic.

2.7 Event-Driven Communication

Communication between services is asynchronous.

Instead of directly invoking downstream services, business events are published to Kafka topics.

Each service independently consumes the required events and performs its business logic.

This approach provides the following advantages:

- Reduced coupling
- Independent scaling
- High throughput
- Better fault tolerance
- Improved reliability
- Event replay capability

2.8 Kafka Topic Flow

The following Kafka topics are used for communication.

Kafka Topic	Producer	Consumer
upi.transactions.initiated	PSP Switch	NPCI Router, PSP Ledger, Audit Service
upi.npci.verified	NPCI Router	CBS Service, NPCI Ledger, Audit Service
upi.cbs.debit.confirm	CBS Service	Bank Ledger, Audit Service
upi.cbs.credit.confirm	CBS Service	Bank Ledger, Audit Service
upi.cbs.reversal	NPCI Router	CBS Service, Audit Service

The Ledger Services consume events independently without affecting the transaction execution flow. This design ensures that ledger persistence does not delay payment processing.

2.9 Database Ownership

Each service writes only to the tables it owns.

Service	Database Table
PSP Ledger Service	psp_ledger
NPCI Ledger Service	npci_ledger
Bank Ledger Service	bank_ledger
Audit Service	audit_log

This ownership model prevents concurrent modifications from multiple services and simplifies data maintenance.

2.10 Design Advantages

The selected architecture offers several advantages over a traditional monolithic implementation.

- Independent deployment of services
- Better scalability under high transaction loads
- Improved fault isolation
- Simplified debugging through Audit Service
- Independent ledger maintenance
- Asynchronous communication using Kafka
- Easy integration of new services
- High maintainability
- Better monitoring capabilities
- Support for horizontal scaling

CHAPTER 3 – APACHE KAFKA

INTEGRATION

3.1 Introduction

Apache Kafka serves as the messaging backbone of the APEX UPI system. It enables asynchronous communication between microservices, allowing each service to process transactions independently without relying on direct service-to-service communication. This event-driven architecture improves scalability, reliability, and fault tolerance while ensuring smooth transaction processing under high loads.

In the proposed system, Kafka acts as the central event bus through which transaction events are exchanged between the PSP Switch, NPCI Router, CBS Service, Ledger Services, and Audit Service.

3.2 Role of Apache Kafka

Kafka is responsible for transmitting transaction events between different services involved in the UPI payment lifecycle.

The responsibilities of Kafka include:

- Asynchronous message delivery
- Decoupling producer and consumer services
- Reliable event distribution
- High-throughput message processing
- Event persistence until consumption
- Support for multiple consumers of the same event

Instead of directly invoking downstream services, each service publishes an event to Kafka. Interested services subscribe to the relevant topic and process the event independently.

3.3 Kafka Components

The implementation consists of the following Kafka components.

Component	Description
Producer	Publishes transaction events
Broker	Stores and distributes messages
Topic	Logical channel for message exchange
Consumer	Reads and processes events
Consumer Group	Enables parallel message processing

3.4 Kafka Topics

The following Kafka topics are used throughout the system.

Kafka Topic	Producer	Consumer(s)
upi.transactions.initiated	PSP Switch	NPCI Router, PSP Ledger Service, Audit Service
upi.npci.verified	NPCI Router	CBS Service, NPCI Ledger Service, Audit Service
upi.cbs.debit.confirm	CBS Service	Bank Ledger Service, Audit Service
upi.cbs.credit.confirm	CBS Service	Bank Ledger Service, Audit Service

upi.cbs.reversal NPCI Router CBS Service, Audit Service

Each topic represents a specific stage of transaction processing.

3.5 Producer Services

The producer services generate Kafka events whenever a transaction reaches a new processing stage.

Producer Service	Published Topic
PSP Switch	upi.transactions.initiated
NPCI Router	upi.npci.verified
CBS Service	upi.cbs.debit.confirm
CBS Service	upi.cbs.credit.confirm
NPCI Router	upi.cbs.reversal

Each producer serializes transaction details into JSON before publishing the message.

3.6 Consumer Services

Each consumer subscribes to one or more Kafka topics.

Consumer Service	Subscribed Topics
NPCI Router	upi.transactions.initiated
CBS Service	upi.npci.verified
PSP Ledger Service	upi.transactions.initiated
NPCI Ledger Service	upi.npci.verified
Bank Ledger Service	upi.cbs.debit.confirm, upi.cbs.credit.confirm
Audit Service	All transaction topics

Consumers process messages independently without affecting other services.

3.7 Message Format

Kafka messages are exchanged in JSON format.

PSP Transaction Event

```
{
  "txnId": "TXN1001",
  "payer": "alice@sbi",
  "payee": "bob@hdfc",
  "amount": 500
}
```

NPCI Verification Event

```
{  
  
  "txnId": "TXN1001",  
  
  "sourceBank": "SBI",  
  
  "destinationBank": "HDFC",  
  
  "amount": 500  
}
```

Debit Confirmation Event

```
{  
  
  "txnId": "TXN1001",  
  
  "account": "alice@sbi",  
  
  "entryType": "DEBIT",  
  
  "amount": 500  
}
```

Credit Confirmation Event

```
{  
  
  "txnId": "TXN1001",  
  
  "account": "bob@hdfc",  
  
  "entryType": "CREDIT",  
  
  "amount": 500  
}
```

Audit Event

```
{  
  
  "txnId": "TXN1001",
```

```

"source": "CBS",

"status": "SUCCESS",

"payer": "alice@sbi",

"payee": "bob@hdfc",

"amount": 500,

"stage": "DEBIT",

"remarks": "Debit completed successfully"

}

```

3.8 Benefits of Kafka Integration

The integration of Apache Kafka provides several advantages to the proposed system.

- Loose coupling between services
- Asynchronous transaction processing
- Improved scalability
- High throughput
- Fault tolerance
- Easy integration of additional consumers
- Better system maintainability
- Real-time event processing
- Independent service deployment

These advantages make Kafka suitable for high-volume payment systems where multiple services must process the same transaction event simultaneously.

3.9 Summary

Apache Kafka forms the communication backbone of the APEX UPI platform. All major services exchange transaction events through Kafka topics rather than direct service calls. This event-driven architecture enables independent processing by Ledger Services and the Audit Service while ensuring reliable, scalable, and fault-tolerant transaction processing. The use of dedicated topics for each transaction stage simplifies system integration and supports future expansion of the platform.

CHAPTER 4 – PSP SWITCH LOW LEVEL DESIGN

4.1 Introduction

The PSP (Payment Service Provider) Switch acts as the entry point of the APEX UPI payment system. It is responsible for receiving payment requests from users, validating the request details, generating a unique transaction identifier, and initiating the payment workflow. Instead of directly communicating with downstream services, the PSP Switch publishes transaction events to Apache Kafka, enabling asynchronous processing.

The PSP Switch is implemented as a Spring Boot microservice following a layered architecture consisting of Controller, Service, Repository, and Configuration layers.

4.2 Module Responsibilities

The PSP Switch performs the following functions:

- Receives payment requests through REST APIs.
- Validates payer and payee information.
- Generates a unique transaction identifier.
- Creates a transaction object.
- Publishes the transaction event to Kafka.
- Returns an acknowledgement to the client.
- Logs transaction information for monitoring.

4.3 Internal Architecture

The PSP Switch follows a layered architecture to separate business logic from data access and presentation.

Layer	Responsibility
Controller	Handles incoming REST requests
Service	Implements payment processing logic
Kafka Producer	Publishes transaction events
Configuration	Stores application properties

This modular structure improves maintainability and simplifies future enhancements.

4.4 Request Processing

When a payment request is received, the Controller forwards the request to the Service layer. The Service validates the request, generates a transaction ID, constructs the transaction payload, and publishes it to the **upi.transactions.initiated** Kafka topic. A success response containing the transaction ID is then returned to the client.

4.5 REST API

The PSP Switch exposes a REST endpoint to initiate UPI payments.

Method	Endpoint	Description
POST	/api/pay	Initiates a UPI transaction

Sample Request

```
{
  "payerVpa": "alice@sbi",
```



```
"payeeVpa":"bob@hdfc",
"amount":500
}
```

Sample Response

```
{
  "txnId":"TXN1001",
  "status":"INITIATED"
}
```

4.6 Kafka Integration

The PSP Switch acts as a Kafka Producer.

Topic	Purpose
upi.transactions.initiated	Publishes payment initiation events

The transaction event is serialized into JSON format before being sent to Kafka.

4.7 Transaction Processing Logic

The payment request undergoes the following processing steps:

- Receive payment request.
- Validate mandatory fields.
- Generate a unique transaction identifier.
- Create transaction payload.
- Publish event to Kafka.
- Return acknowledgement.

Each transaction is assigned a unique identifier, which is used throughout the payment lifecycle for tracking and auditing.

4.8 Exception Handling

The PSP Switch handles common runtime exceptions to ensure reliable processing.

Exception	Handling
Invalid request	Return HTTP 400 Bad Request
Kafka unavailable	Log error and return failure response
Unexpected exception	Log exception and return HTTP 500

Error details are recorded in the application logs for troubleshooting.

4.9 Logging

The PSP Switch logs key events during processing, including:

- Incoming payment requests.
- Generated transaction IDs.
- Kafka publishing status.
- Validation failures.
- System exceptions.

These logs assist in monitoring and debugging transaction execution.

4.10 Design Advantages

The implementation provides the following benefits:

- Layered and modular architecture.
- Asynchronous event publishing using Kafka.
- Loose coupling between services.
- Easy integration with downstream components.
- Improved scalability and maintainability.
- Simplified monitoring and debugging.

4.11 Summary

The PSP Switch is the entry point of the payment ecosystem and is responsible for initiating the transaction workflow. It validates incoming requests, generates transaction identifiers, and publishes events to Kafka for further processing. Its modular implementation using Spring Boot and asynchronous messaging ensures reliable, scalable, and maintainable transaction initiation.

CHAPTER 5 – NPCI ROUTER LOW LEVEL DESIGN

5.1 Introduction

The NPCI Router is the central routing component of the APEX UPI system. It receives transaction events published by the PSP Switch, validates the transaction, identifies the payer and beneficiary banks, and forwards the transaction to the Core Banking System (CBS) for processing. It also coordinates transaction status updates and initiates reversal requests when necessary.

The NPCI Router is implemented as a Spring Boot microservice and communicates with other services using Apache Kafka.

5.2 Module Responsibilities

The NPCI Router performs the following responsibilities:

- Consumes payment initiation events from Kafka.
- Validates transaction details.
- Identifies payer and beneficiary banks.
- Routes the transaction to the CBS Service.
- Publishes transaction verification events.
- Handles transaction failures.
- Initiates reversal requests when required.
- Logs routing activities.

5.3 Internal Architecture

The NPCI Router follows a layered architecture similar to other microservices.

Layer	Responsibility
Kafka Consumer	Receives payment events
Service	Performs routing and validation logic

Kafka Producer	Publishes verification and reversal events
----------------	--

Configuration	Stores Kafka and application configuration
---------------	--

This structure separates business logic from communication components and improves maintainability.

5.4 Kafka Integration

The NPCI Router functions as both a Kafka Consumer and Producer.

Consumer Topics

Topic	Purpose
upi.transactions.initiated	Receives payment initiation requests

Producer Topics

Topic	Purpose
upi.npci.verified	Publishes verified transaction details
upi.cbs.reversal	Publishes reversal requests when required

5.5 Transaction Validation

After receiving a transaction, the NPCI Router validates the request before forwarding it to the CBS Service.

Validation includes:

- Transaction ID availability.
- Payer bank identification.
- Beneficiary bank identification.
- Mandatory field validation.
- Transaction format verification.

If validation succeeds, the transaction is forwarded to the next stage.

5.6 Routing Logic

The routing logic determines the destination bank based on the beneficiary account information.

The routing process consists of:

- Identifying the payer bank.
- Identifying the beneficiary bank.
- Creating the routing request.
- Publishing the verified transaction event.

This approach ensures that only valid transactions reach the banking system.

5.7 Reversal Handling

If the CBS Service reports a transaction failure or credit operation cannot be completed successfully, the NPCI Router generates a reversal request.

The reversal event is published to the **upi.cbs.reversal** Kafka topic, allowing the CBS Service to reverse the debit transaction and restore the payer's account balance.

This mechanism helps maintain transaction consistency and prevents partial transaction completion.

5.8 Exception Handling

The NPCI Router handles common exceptions during routing and verification.

Exception	Handling
Invalid transaction	Reject transaction and log error
Kafka communication failure	Retry publishing and log failure
Routing error	Generate failure response
Unexpected exception	Log error for debugging

5.9 Logging

The NPCI Router records important routing events, including:

- Transaction received.
- Validation status.
- Routing completion.
- Kafka publishing status.
- Reversal initiation.
- Processing errors.

These logs support monitoring and troubleshooting of transaction processing.

5.10 Design Advantages

The implementation offers the following advantages:

- Centralized transaction routing.
- Asynchronous communication using Kafka.
- Reduced dependency between services.
- Improved fault handling through reversal events.
- Modular and maintainable implementation.
- Scalable routing architecture.

5.11 Summary

The NPCI Router serves as the central coordinator of the UPI transaction workflow. It validates payment requests, routes them to the appropriate banking service, and manages transaction verification and reversal events. By leveraging Apache Kafka for communication, the NPCI Router enables reliable, scalable, and loosely coupled transaction processing while ensuring accurate routing between participating banks.

CHAPTER 6 – CBS SERVICE LOW LEVEL DESIGN

6.1 Introduction

The Core Banking System (CBS) Service is responsible for executing financial transactions within the APEX UPI platform. It performs debit and credit operations on customer accounts after receiving verified transaction requests from the NPCI Router. The CBS Service also handles transaction reversals whenever a transaction fails during processing.

The CBS Service is implemented as a Spring Boot microservice and communicates with other services through Apache Kafka.

6.2 Module Responsibilities

The CBS Service performs the following functions:

- Consumes verified transaction events from Kafka.
- Validates account information.
- Debits the payer account.
- Credits the beneficiary account.
- Handles transaction reversal requests.
- Publishes debit confirmation events.
- Publishes credit confirmation events.
- Updates transaction status.
- Logs transaction activities.

6.3 Internal Architecture

The CBS Service follows a layered architecture.

Layer	Responsibility
Kafka Consumer	Receives verified and reversal events

Service	Executes banking operations
Repository	Performs database operations
Kafka Producer	Publishes transaction confirmation events
Configuration	Stores application configurations

This architecture separates communication, business logic, and persistence, making the service easier to maintain and extend.

6.4 Kafka Integration

The CBS Service functions as both a Kafka Consumer and Producer.

Consumer Topics

Topic	Purpose
upi.npci.verified	Receives verified transactions
upi.cbs.reversal	Receives reversal requests

Producer Topics

Topic	Purpose
upi.cbs.debit.confirm	Debit completed successfully

upi.cbs.credit.confirm Credit completed successfully

6.5 Debit Processing

Upon receiving a verified transaction, the CBS Service first processes the debit operation.

The debit process includes:

- Validate payer account.
- Verify available balance.
- Debit the transaction amount.
- Update account balance.
- Publish debit confirmation event.

If the debit operation fails, the transaction is terminated and an error is logged.

6.6 Credit Processing

After a successful debit, the CBS Service credits the beneficiary account.

The credit process includes:

- Validate beneficiary account.
- Credit the transaction amount.
- Update beneficiary balance.
- Publish credit confirmation event.

Successful completion of both debit and credit operations marks the transaction as completed.

6.7 Reversal Processing

If a transaction cannot be completed after the debit operation, a reversal request is received from the NPCI Router.

The CBS Service performs the following actions:

- Validate reversal request.
- Identify original transaction.
- Restore amount to payer account.
- Update transaction status.
- Record reversal activity.

This ensures that no customer loses funds due to incomplete transactions.

6.8 Transaction Status

During processing, the CBS Service maintains different transaction states.

Status	Description
VERIFIED	Transaction received from NPCI
DEBIT SUCCESS	Amount debited successfully
CREDIT SUCCESS	Amount credited successfully
COMPLETED	Transaction completed
REVERSED	Amount restored after failure
FAILED	Transaction could not be processed

6.9 Exception Handling

The CBS Service manages runtime failures using structured exception handling.

Exception	Handling
Account not found	Reject transaction

Insufficient balance	Stop processing
Credit failure	Initiate reversal
Kafka failure	Log and retry
Unexpected exception	Record error and terminate processing

Proper exception handling prevents inconsistent account updates.

6.10 Logging

The CBS Service records operational logs throughout transaction execution.

The following activities are logged:

- Transaction received.
- Debit initiated.
- Debit completed.
- Credit initiated.
- Credit completed.
- Reversal initiated.
- Reversal completed.
- Processing failures.

These logs assist in debugging and monitoring transaction execution.

6.11 Design Advantages

The CBS Service offers several advantages:

- Independent banking operations.
- Reliable debit and credit processing.
- Automatic reversal handling.
- Event-driven communication.
- Modular implementation.
- High maintainability.

6.12 Summary

The CBS Service is responsible for executing the financial operations of the UPI transaction lifecycle. It performs debit, credit, and reversal operations while communicating asynchronously through Apache Kafka. By publishing confirmation events after each successful operation, the CBS Service enables the Ledger Engine and Audit Service to maintain accurate financial records and transaction history without affecting the core banking workflow.

CHAPTER 7 – PSP LEDGER SERVICE LOW LEVEL DESIGN

7.1 Introduction

The PSP Ledger Service is responsible for maintaining the payment initiation records generated by the PSP Switch. It receives transaction initiation events from Apache Kafka and stores them in the PostgreSQL database. The service provides a persistent record of every transaction initiated through the PSP Switch, enabling transaction tracking and future reconciliation.

The service is implemented as an independent Spring Boot microservice using Spring Data JPA for database operations and Apache Kafka for asynchronous event consumption.

7.2 Module Responsibilities

The PSP Ledger Service performs the following responsibilities:

- Consumes transaction initiation events from Kafka.
- Parses incoming JSON messages.
- Validates transaction data.
- Creates ledger records.
- Stores transaction details in PostgreSQL.
- Maintains transaction history.
- Logs successful and failed operations.

7.3 Internal Architecture

The service follows a standard Spring Boot layered architecture.

Layer	Responsibility
Kafka Consumer	Receives transaction events
Service	Processes ledger information

Repository	Performs database operations
Entity	Maps ledger data to PostgreSQL table
Configuration	Stores application properties

7.4 Kafka Integration

The PSP Ledger Service acts only as a Kafka Consumer.

Consumer Topic	Purpose
upi.transactions.initiated	Receives payment initiation events

The service does not publish messages to Kafka. Its responsibility is limited to storing ledger information.

7.5 Transaction Processing

When a transaction initiation event is received, the service performs the following operations:

- Receive Kafka message.
- Convert JSON data into a transaction object.
- Extract transaction details.
- Create a PSP ledger record.
- Save the record in PostgreSQL.
- Log successful persistence.

Each transaction is stored independently, ensuring a permanent record of payment initiation.

7.6 Database Interaction

The service stores transaction details in the **psp_ledger** table.

The following information is recorded:

- Transaction ID
- Payer VPA
- Payee VPA
- Transaction Amount
- Transaction Status

Database operations are performed using Spring Data JPA, eliminating the need for manual SQL queries.

7.7 Entity Design

The PSP Ledger entity represents a single payment initiation record.

The entity contains the following fields:

- ID
- Transaction ID
- Payer VPA
- Payee VPA
- Amount
- Status

Each entity object corresponds to one row in the PostgreSQL table.

7.8 Repository Design

The repository layer extends Spring Data JPA Repository to perform CRUD operations.

The repository is responsible for:

- Saving ledger records.
- Retrieving transaction details.
- Supporting future search operations.
- Abstracting database interactions from business logic.

7.9 Exception Handling

The service handles common runtime exceptions during transaction persistence.

Exception	Handling
Invalid Kafka message	Ignore and log error
JSON parsing failure	Log parsing exception
Database failure	Log persistence error
Unexpected exception	Record stack trace for debugging

This prevents service termination due to malformed messages.

7.10 Logging

The service records important processing events.

The following activities are logged:

- Kafka message received.
- Transaction ID processed.
- Ledger entry stored.
- Database errors.
- Message parsing failures.

These logs simplify transaction tracing and troubleshooting.

7.11 Design Advantages

The PSP Ledger Service offers the following benefits:

- Independent transaction persistence.
- Loose coupling with other services.
- Asynchronous event processing.
- Simplified transaction tracking.
- Improved maintainability.
- Easy scalability.

7.12 Summary

The PSP Ledger Service maintains a persistent record of all payment initiation events received from the PSP Switch. By consuming Kafka messages and storing them in PostgreSQL, the service provides reliable transaction history while remaining completely independent of the transaction processing workflow. This design improves traceability and supports future reconciliation and reporting requirements.

CHAPTER 8 – NPCI LEDGER SERVICE

LOW LEVEL DESIGN

8.1 Introduction

The NPCI Ledger Service is responsible for maintaining transaction routing and verification records generated by the NPCI Router. It consumes verified transaction events from Apache Kafka and stores them in PostgreSQL. The service maintains a permanent record of routing information, enabling transaction tracking, reconciliation, and operational analysis.

The service is implemented as an independent Spring Boot microservice using Apache Kafka for event consumption and Spring Data JPA for database persistence.

8.2 Module Responsibilities

The NPCI Ledger Service performs the following functions:

- Consumes transaction verification events from Kafka.
- Processes routing information.
- Validates incoming transaction data.
- Stores transaction details in PostgreSQL.
- Maintains verification history.
- Logs successful and failed database operations.

8.3 Internal Architecture

The service follows a layered architecture.

Layer	Responsibility
Kafka Consumer	Receives verification events
Service	Processes transaction data

Repository	Stores records in PostgreSQL
Entity	Represents the database table
Configuration	Stores application configuration

This architecture promotes modularity and simplifies maintenance.

8.4 Kafka Integration

The NPCI Ledger Service functions only as a Kafka Consumer.

Consumer Topic	Purpose
upi.npci.verified	Receives verified transaction events

The service does not publish Kafka messages. It only records routing information.

8.5 Transaction Processing

After receiving a verification event, the service performs the following operations:

- Consume Kafka message.
- Parse JSON transaction data.
- Extract routing information.
- Create NPCI ledger record.
- Store the record in PostgreSQL.
- Log successful completion.

Each verification event results in a single ledger entry.

8.6 Database Interaction

The service stores transaction details in the **npci_ledger** table.

The following information is maintained:

- Transaction ID
- Source Bank
- Destination Bank
- Transaction Amount
- Verification Status

Spring Data JPA manages all database operations, ensuring efficient interaction with PostgreSQL.

8.7 Entity Design

The NPCI Ledger entity represents a verified transaction.

Each entity contains:

- ID
- Transaction ID
- Source Bank
- Destination Bank
- Amount
- Status

Each record corresponds to one successfully verified transaction.

8.8 Repository Design

The repository layer provides database access using Spring Data JPA.

The repository is responsible for:

- Saving ledger records.
- Retrieving transaction information.
- Supporting future reporting requirements.
- Abstracting database operations from business logic.

8.9 Exception Handling

The service includes exception handling to improve reliability.

Exception	Handling
Invalid Kafka message	Log and ignore message
JSON parsing error	Record parsing failure
Database exception	Log persistence error
Unexpected exception	Record application error

This prevents interruptions during transaction processing.

8.10 Logging

The following activities are recorded in the application logs:

- Verification event received.
- Transaction processed.
- Ledger record stored.
- Database operation status.
- Parsing errors.
- System exceptions.

These logs assist in system monitoring and troubleshooting.

8.11 Design Advantages

The NPCI Ledger Service provides several advantages:

- Independent routing history.
- Reliable transaction persistence.
- Loose coupling with business services.

- Asynchronous processing using Kafka.
- Simplified reconciliation.
- Improved maintainability.

8.12 Summary

The NPCI Ledger Service maintains a permanent record of transaction verification and routing information. By consuming verification events from Kafka and storing them in PostgreSQL, the service provides reliable transaction traceability while remaining independent of the payment execution process. Its modular implementation supports scalability, maintainability, and future system enhancements.

CHAPTER 9 – BANK LEDGER SERVICE

LOW LEVEL DESIGN

9.1 Introduction

The Bank Ledger Service is responsible for maintaining the accounting records generated after successful banking operations. It consumes debit and credit confirmation events published by the CBS Service and stores them in the PostgreSQL database. The service ensures that every successful transaction is recorded as separate debit and credit entries, providing a complete financial record for reconciliation and reporting.

The Bank Ledger Service is implemented as an independent Spring Boot microservice using Apache Kafka for event consumption and Spring Data JPA for database persistence.

9.2 Module Responsibilities

The Bank Ledger Service performs the following responsibilities:

- Consumes debit confirmation events.
- Consumes credit confirmation events.
- Processes transaction details.
- Creates debit ledger entries.
- Creates credit ledger entries.
- Stores accounting records in PostgreSQL.
- Maintains transaction history.
- Logs successful and failed operations.

9.3 Internal Architecture

The service follows the standard layered architecture adopted throughout the project.

Layer	Responsibility
Kafka Consumer	Receives debit and credit events
Service	Processes ledger information

Repository	Stores records in PostgreSQL
Entity	Represents the ledger table
Configuration	Stores application configuration

This architecture ensures clear separation between event processing, business logic, and data persistence.

9.4 Kafka Integration

The Bank Ledger Service functions as a Kafka Consumer.

Consumer Topics

Topic	Purpose
upi.cbs.debit.confirm	Receives successful debit events
upi.cbs.credit.confirm	Receives successful credit events

The service does not publish any Kafka messages.

9.5 Debit Entry Processing

When a debit confirmation event is received from the CBS Service, the Bank Ledger Service performs the following operations:

- Receive Kafka message.
- Parse transaction details.
- Extract payer account information.
- Create a debit ledger entry.
- Store the record in PostgreSQL.
- Log successful persistence.

9.6 Credit Entry Processing

After receiving a credit confirmation event, the service performs the following operations:

- Receive Kafka message.
- Parse transaction details.
- Extract beneficiary account information.
- Create a credit ledger entry.
- Store the record in PostgreSQL.
- Log successful persistence.

Each successful credit operation generates one ledger record.

9.7 Double-Entry Recording

The Bank Ledger Service follows the principle of double-entry accounting.

For every successful UPI transaction:

- One **DEBIT** entry is created for the payer account.
- One **CREDIT** entry is created for the beneficiary account.

Example:

Transaction ID	Account	Entry Type	Amount
TXN1001	alice@sbi	DEBIT	₹500
TXN1001	bob@hdfc	CREDIT	₹500

This approach ensures accurate financial record keeping and simplifies reconciliation.

9.8 Database Interaction

The service stores all records in the **bank_ledger** table.

Each ledger record contains:

- Transaction ID

- Account
- Entry Type
- Transaction Amount
- Status

Spring Data JPA is used to perform all database operations.

9.9 Entity Design

The Bank Ledger entity represents a single accounting entry.

The entity contains the following fields:

- ID
- Transaction ID
- Account
- Entry Type
- Amount
- Status

Each row in the database represents either a debit or a credit entry.

9.10 Repository Design

The repository layer manages communication with PostgreSQL.

Its responsibilities include:

- Saving ledger records.
- Retrieving transaction details.
- Supporting future reporting and reconciliation.
- Abstracting database access from business logic.

9.11 Exception Handling

The service handles runtime exceptions during message processing and persistence.

Exception	Handling
Invalid Kafka message	Ignore and log error

JSON parsing failure	Record parsing error
Database exception	Log persistence failure
Unexpected exception	Record application error

These mechanisms ensure uninterrupted processing of subsequent messages.

9.12 Logging

The Bank Ledger Service records the following events:

- Debit confirmation received.
- Credit confirmation received.
- Ledger record stored.
- Database operation status.
- Parsing failures.
- System exceptions.

Logging assists in transaction tracing and operational monitoring.

9.13 Design Advantages

The implementation provides several advantages:

- Independent accounting records.
- Separate debit and credit entries.
- Asynchronous processing through Kafka.
- Reliable transaction persistence.
- Simplified reconciliation.
- Improved maintainability.
- Scalable microservice architecture.

9.14 Summary

The Bank Ledger Service maintains the financial accounting records of the UPI transaction lifecycle. It consumes debit and credit confirmation events from the CBS Service and stores them as separate ledger entries in PostgreSQL.

CHAPTER 10 – AUDIT SERVICE LOW LEVEL DESIGN

10.1 Introduction

The Audit Service is responsible for maintaining a centralized audit trail of every transaction processed within the APEX UPI system. Unlike the Ledger Services, which maintain specific financial records, the Audit Service records the complete transaction lifecycle from initiation to completion. It continuously monitors Kafka events generated by different microservices and stores them in PostgreSQL for monitoring, debugging, compliance, and transaction analysis.

The Audit Service is implemented as an independent Spring Boot microservice using Apache Kafka for event consumption and Spring Data JPA for database persistence.

10.2 Module Responsibilities

The Audit Service performs the following responsibilities:

- Consumes transaction events from multiple Kafka topics.
- Processes transaction information.
- Maintains the complete transaction history.
- Stores audit records in PostgreSQL.
- Records transaction status changes.
- Logs successful and failed operations.
- Supports transaction tracing and troubleshooting.

10.3 Internal Architecture

The Audit Service follows the standard layered architecture.

Layer	Responsibility
Kafka Consumer	Receives transaction events
Service	Processes audit information

Repository	Stores audit records
Entity	Maps audit data to PostgreSQL
Configuration	Stores application configuration

This architecture keeps event processing independent from database operations.

10.4 Kafka Integration

The Audit Service functions exclusively as a Kafka Consumer.

Consumer Topics

Topic	Purpose
upi.transactions.initiated	Records payment initiation
upi.npci.verified	Records transaction verification
upi.cbs.debit.confirm	Records successful debit
upi.cbs.credit.confirm	Records successful credit
upi.cbs.reversal	Records transaction reversal

Unlike other services, the Audit Service listens to multiple topics to capture the complete transaction lifecycle.

10.5 Audit Processing

Whenever a transaction event is received, the Audit Service performs the following operations:

- Receive Kafka message.
- Parse transaction information.
- Identify the event source.
- Extract transaction details.
- Create an audit record.
- Store the record in PostgreSQL.
- Log successful persistence.

Each Kafka event results in one audit record.

10.6 Database Interaction

The Audit Service stores transaction information in the **audit_log** table.

Each audit record contains:

- Transaction ID
- Event Source
- Transaction Status
- Payer
- Payee
- Amount
- Processing Stage
- Remarks
- Timestamp

The service uses Spring Data JPA for all database operations.

10.7 Entity Design

The Audit entity represents one transaction event.

The entity contains the following fields:

- ID
- Transaction ID
- Source
- Status
- Payer
- Payee

- Amount
- Stage
- Remarks
- Created Timestamp

Each event received from Kafka generates a new record in the audit table.

10.8 Repository Design

The repository layer manages database operations using Spring Data JPA.

Its responsibilities include:

- Saving audit records.
- Retrieving transaction history.
- Supporting future reporting requirements.
- Providing abstraction from SQL operations.

10.9 Transaction Traceability

The Audit Service enables complete transaction tracking.

For a successful transaction, the audit table may contain records such as:

Stage	Status
INITIATION	INITIATED
ROUTING	VERIFIED
DEBIT	SUCCESS
CREDIT	SUCCESS

If a transaction fails, an additional **REVERSAL** record is created, allowing administrators to understand the complete sequence of events.

10.10 Exception Handling

The Audit Service handles runtime exceptions during event processing.

Exception	Handling
Invalid Kafka message	Ignore and log error
JSON parsing failure	Record parsing error
Database exception	Log persistence failure
Unexpected exception	Log application error

Exception handling ensures continuous monitoring even when malformed messages are received.

10.11 Logging

The Audit Service records the following operational events:

- Kafka message received.
- Audit record created.
- Database operation status.
- Message parsing failures.
- Persistence errors.
- System exceptions.

These logs assist developers and administrators in monitoring system health.

10.12 Design Advantages

The Audit Service provides the following benefits:

- Centralized transaction monitoring.
- Complete transaction lifecycle history.
- Simplified debugging.

- Improved regulatory compliance.
- Independent event processing.
- Reliable audit persistence.
- Better operational visibility.

10.13 Summary

The Audit Service acts as the centralized monitoring component of the APEX UPI platform. By consuming transaction events from all major Kafka topics and storing them in PostgreSQL, it provides complete transaction traceability throughout the payment lifecycle. Its independent microservice architecture ensures that auditing activities do not affect transaction execution while supporting monitoring, debugging, compliance, and future reporting requirements.

CHAPTER 11 – REDIS INTEGRATION

11.1 Introduction

Redis is integrated into the APEX UPI system to improve application performance by providing fast in-memory data access. Frequently accessed or short-lived transaction information can be stored in Redis, reducing repeated database queries and improving response time during transaction processing.

Redis acts as a temporary storage layer and complements PostgreSQL, which serves as the permanent database.

11.2 Purpose

The primary objectives of integrating Redis are:

- Improve application performance.
- Reduce database load.
- Store temporary transaction data.
- Enable faster data retrieval.
- Support high-concurrency transaction processing.

11.3 Redis Architecture

Redis is deployed as an independent service and communicates with the Spring Boot microservices over the network.

The application accesses Redis using the Spring Data Redis framework, allowing data to be stored and retrieved efficiently through cache operations.

11.4 Data Storage

Redis stores temporary information such as:

- Transaction cache
- Session data
- Frequently accessed records
- Short-lived processing data

Since Redis is an in-memory database, cached information can be accessed much faster than relational database records.

11.5 Integration with Spring Boot

Redis is integrated using Spring Data Redis.

The application configuration includes:

- Redis Host
- Redis Port
- Connection Factory
- Redis Template

These configurations establish communication between the microservices and the Redis server.

11.6 Cache Operations

The common cache operations performed by the application include:

- Store data
- Retrieve cached data
- Update cached records
- Remove expired entries

These operations help improve the overall performance of the application.

11.7 Advantages

Redis integration provides the following benefits:

- High-speed data access.
- Reduced PostgreSQL workload.
- Improved application performance.
- Low latency.
- Better scalability.
- Efficient handling of concurrent requests.

11.8 Summary

Redis serves as the caching layer within the APEX UPI platform by providing fast in-memory access to temporary data. Its integration with Spring Boot improves application performance, reduces database load, and enhances the overall efficiency of transaction processing while PostgreSQL remains the primary persistent storage.

CHAPTER 12 – REST API IMPLEMENTATION

12.1 Introduction

REST APIs provide the interface through which external applications and users interact with the APEX UPI system. The APIs are implemented using the Spring Boot framework and enable secure communication between the client application and backend services. The PSP Switch exposes the primary APIs for transaction initiation, while internal microservices communicate mainly through Apache Kafka.

12.2 API Architecture

The REST APIs follow the standard Spring Boot layered architecture.

Layer	Responsibility
Controller	Receives HTTP requests and returns responses
Service	Implements business logic
Repository	Performs database operations where applicable

The Controller layer validates incoming requests and delegates processing to the Service layer. The Service layer executes business logic and returns appropriate responses to the client.

12.3 HTTP Methods

The application uses standard HTTP methods for communication.

Method	Purpose
--------	---------

POST Create a new transaction

GET Retrieve transaction details

PUT Update transaction information (if required)

12.4 API Request Processing

The API processing follows the sequence below:

- Client sends HTTP request.
- Controller validates request parameters.
- Service processes business logic.
- Transaction event is published to Kafka.
- Response is returned to the client.

This approach keeps API processing lightweight while allowing downstream services to process transactions asynchronously.

12.5 Sample API

Initiate Payment

Property	Value
Method	POST
Endpoint	<code>/api/pay</code>
Description	Initiates a UPI transaction

Sample Request

```
{  
  "payerVpa": "alice@sbi",  
  "payeeVpa": "bob@hdfc",  
  "amount": 500  
}
```

Sample Response

```
{  
  "txnId": "TXN1001",  
  "status": "INITIATED",  
  "message": "Transaction initiated successfully"  
}
```

12.6 Response Codes

HTTP Code	Description
200	Request processed successfully
201	Transaction created successfully
400	Invalid request
404	Resource not found

500 Internal server error

12.7 Request Validation

The application validates incoming requests before processing.

Validation includes:

- Mandatory field verification.
- Valid payer and payee information.
- Positive transaction amount.
- Proper request format.

Invalid requests are rejected with appropriate HTTP error responses.

12.8 Exception Handling

REST APIs handle runtime exceptions gracefully.

Exception	Action
Invalid input	Return HTTP 400
Processing failure	Return HTTP 500
Unexpected exception	Log error and return failure response

12.9 API Advantages

The REST API implementation provides:

- Simple client interaction.
- Platform-independent communication.
- Easy integration with external applications.

- Standard HTTP-based communication.
- Scalable and maintainable architecture.

12.10 Summary

The REST APIs provide the entry point for transaction initiation in the APEX UPI platform. Built using Spring Boot, they validate client requests, invoke business services, and trigger asynchronous transaction processing through Apache Kafka. This design ensures efficient request handling while keeping the APIs simple, scalable, and easy to integrate with client applications.

CHAPTER 13 – VIRTUAL MACHINE DEPLOYMENT

13.1 Introduction

The APEX UPI system is deployed using multiple Virtual Machines (VMs), where each VM hosts a dedicated service responsible for a specific stage of the UPI transaction lifecycle. This deployment approach provides better service isolation, independent execution, simplified monitoring, and easier maintenance.

Each microservice runs independently on its assigned virtual machine and communicates with other services through Apache Kafka.

13.2 Deployment Environment

The project is deployed on multiple virtual machines.

Virtual Machine	Hosted Service
VM-1	PSP Switch
VM-2	NPCI Router
VM-3	CBS Service
VM-4	Ledger Services & Audit Service
VM-5	Load Runner / JMeter

This deployment architecture enables independent execution and monitoring of each service.

13.3 Service Deployment

Each Spring Boot application is packaged as an executable JAR file and deployed on its respective virtual machine.

The deployment process includes:

- Building the application using Maven.
- Copying the JAR file to the target VM.
- Configuring application properties.
- Starting the Spring Boot application.
- Verifying service availability.

Each service runs independently without affecting the execution of other services.

13.4 Communication Between Virtual Machines

The virtual machines communicate through Apache Kafka.

Instead of directly invoking another service, each microservice publishes or consumes Kafka events. This reduces service dependency and enables asynchronous transaction processing.

13.5 Advantages

The Virtual Machine deployment provides the following advantages:

- Independent deployment of services.
- Better fault isolation.
- Easy scalability.
- Simplified maintenance.
- Independent service monitoring.
- Improved reliability.
- Better resource utilization.

13.6 Summary

Deploying the APEX UPI system across multiple virtual machines provides a modular and scalable architecture. Each service operates independently while communicating through Apache Kafka, ensuring reliable transaction processing and simplified system management.

CHAPTER 14 – CONFIGURATION MANAGEMENT

14.1 Introduction

Configuration management is used to maintain application settings required by different microservices. Externalizing configuration allows services to be modified without changing the application source code.

Each microservice contains its own configuration file containing database, Kafka, Redis, and server settings.

14.2 Configuration Files

Each Spring Boot service uses an **application.properties** file.

The configuration includes:

- Server Port
- PostgreSQL Connection
- Kafka Configuration
- Redis Configuration
- JPA Properties
- Logging Configuration

14.3 Database Configuration

The PostgreSQL connection is configured using the following properties:

- Database URL
- Username
- Password
- Driver Class
- Hibernate Configuration

These properties establish communication with the PostgreSQL database.

14.4 Kafka Configuration

Kafka integration is configured using:

- Bootstrap Server

- Consumer Group ID
- Topic Subscription
- Offset Configuration

These settings enable communication with the Kafka broker.

14.5 Configuration Benefits

External configuration provides several advantages:

- Easy maintenance.
- Environment-specific settings.
- Reduced code modification.
- Improved flexibility.
- Simplified deployment.

14.6 Summary

Configuration management enables each microservice to operate independently while maintaining flexible environment-specific settings. Separating configuration from application code improves maintainability and simplifies deployment across multiple virtual machines.

CHAPTER 15 – EXCEPTION HANDLING AND LOGGING

15.1 Introduction

Exception handling and logging are essential for maintaining application reliability and simplifying debugging. Every microservice in the APEX UPI system includes mechanisms to detect runtime errors, handle failures gracefully, and record operational information.

15.2 Exception Handling

The application handles common runtime exceptions during transaction processing.

Examples include:

- Invalid request data.
- Kafka communication failure.
- Database connection failure.
- JSON parsing error.
- Unexpected system exceptions.

Appropriate error responses are generated without interrupting the execution of other services.

15.3 Logging

Each microservice records important execution details.

The application logs include:

- Service startup.
- Incoming requests.
- Kafka message processing.
- Database operations.
- Successful transactions.
- Error messages.
- Exception details.

Logging helps developers monitor application behaviour and troubleshoot failures.

15.4 Logging Benefits

The logging mechanism provides:

- Easier debugging.
- Transaction monitoring.
- Error tracing.
- System health monitoring.
- Operational transparency.

15.5 Summary

Effective exception handling and logging improve the reliability and maintainability of the APEX UPI platform. These mechanisms ensure smooth execution while providing sufficient information for debugging and operational monitoring.

CHAPTER 16 – PERFORMANCE AND SCALABILITY

16.1 Introduction

The APEX UPI system is designed to support high-volume transaction processing while maintaining low response time and high reliability. The microservice architecture and asynchronous communication model improve the overall scalability of the system.

16.2 Performance Features

The following design decisions improve system performance:

- Independent microservices.
- Apache Kafka for asynchronous messaging.
- Redis for fast data access.
- PostgreSQL for reliable persistence.
- Parallel transaction processing.
- Independent deployment on virtual machines.

16.3 Scalability

The system can be scaled by deploying additional instances of individual microservices without affecting other services.

Scalability features include:

- Independent service deployment.
- Event-driven communication.
- Load distribution using Kafka.
- Modular architecture.
- Horizontal service expansion.

16.4 Performance Testing

Apache JMeter is used to evaluate system performance under concurrent transaction loads.

Performance metrics include:

- Response Time

- Throughput
- Error Rate
- Concurrent Users
- Transaction Success Rate

16.5 Summary

The proposed architecture supports scalable and efficient transaction processing through microservices, Kafka, Redis, and virtual machine deployment, making it suitable for handling high transaction volumes.

CHAPTER 17 – SECURITY CONSIDERATIONS

17.1 Introduction

Security is an important aspect of the APEX UPI platform. Appropriate validation and controlled communication between services help ensure secure transaction processing.

17.2 Security Measures

The system incorporates the following security practices:

- Request validation.
- Input sanitization.
- Controlled REST API access.
- Internal communication through Kafka.
- Database authentication.
- Centralized logging.

17.3 Data Protection

Sensitive transaction information is stored securely in PostgreSQL, while temporary data is maintained in Redis when required. Access to services is controlled through application configuration.

17.4 Summary

The proposed architecture provides a secure environment for processing financial transactions while maintaining data integrity and controlled communication between microservices.

CHAPTER 18 – TESTING STRATEGY

18.1 Introduction

Testing is performed to verify the correctness, reliability, and performance of the APEX UPI system. Both functional and performance testing are conducted during development.

18.2 Functional Testing

Functional testing verifies the correctness of individual modules.

The following functionalities are tested:

- Payment initiation.
- Transaction routing.
- Debit processing.
- Credit processing.
- Ledger updates.
- Audit logging.
- Kafka message processing.

18.3 Integration Testing

Integration testing ensures that all microservices communicate correctly through Apache Kafka.

The following integrations are verified:

- PSP Switch → NPCI Router
- NPCI Router → CBS Service
- CBS Service → Ledger Services
- CBS Service → Audit Service

18.4 Performance Testing

Apache JMeter is used to simulate concurrent users and evaluate system performance.

The following metrics are monitored:

- Average Response Time
- Throughput
- Requests per Second
- Error Percentage

- CPU and Memory Usage

18.5 Summary

Testing confirms that the system functions correctly, supports concurrent transaction processing, and maintains reliable communication between all microservices.

CHAPTER 19 – FUTURE ENHANCEMENTS

The proposed system can be extended with additional features to improve functionality and scalability.

Future enhancements include:

- Integration with real banking systems.
- Support for multiple payment modes.
- Enhanced fraud detection using Machine Learning.
- Real-time transaction analytics.
- Distributed database support.
- Kubernetes-based deployment.
- Advanced monitoring dashboards.
- Multi-region deployment.
- Enhanced security mechanisms.
- AI-based anomaly detection.

These enhancements will improve system performance, reliability, and operational efficiency.

CHAPTER 20 – CONCLUSION

The Low Level Design document presented the detailed implementation of the APEX UPI system, covering the internal architecture of each microservice, Kafka integration, REST API implementation, virtual machine deployment, configuration management, exception handling, performance, security, and testing strategy.

The system follows a modular microservice architecture where each service performs a specific responsibility while communicating asynchronously through Apache Kafka. The implementation of dedicated PSP, NPCI, Bank Ledger, and Audit services improves transaction traceability, reliability, and maintainability.

The proposed design provides a scalable, fault-tolerant, and event-driven architecture capable of supporting high-volume UPI transaction processing. It also establishes a strong foundation for future enhancements and integration with enterprise-grade payment systems.